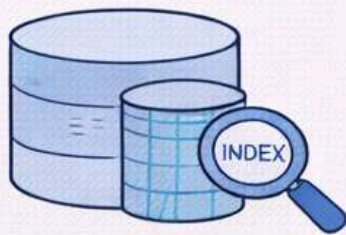


LEARN SQL



```
SELECT name  
FROM users  
WHERE...
```



prepared by Vojasvi Reddy

SQL: From Beginner to Pro

A complete A-Z roadmap to master databases with confidence



- SQL stands for Structured Query Language
- Store, retrieve, manipulate, and analyze data
- SQL is the core language behind modern applications, analytics, and data science
- Used by:
 - Software Engineers
 - Data Analysts
 - Backend Developers
 - Data Scientists



“If data is power, SQL is the key.”

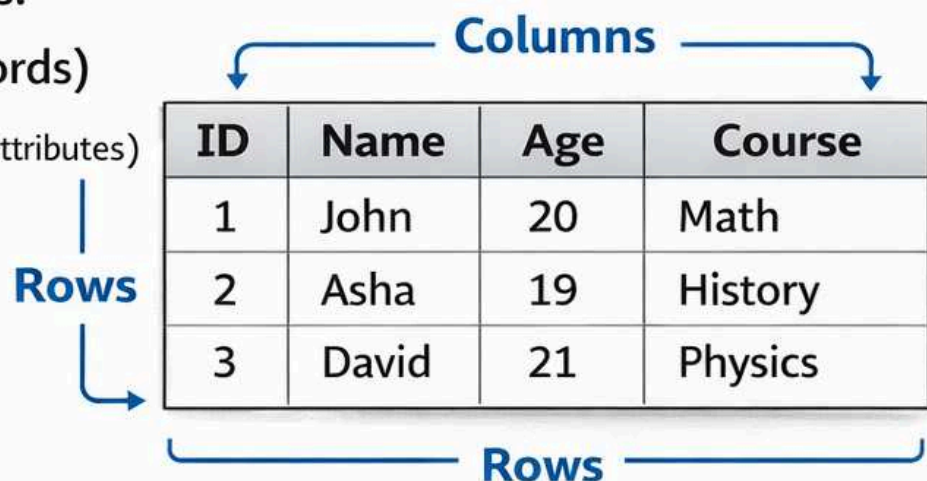
Why Learn SQL?

- Works with almost every database
- Easy to learn, powerful to master
- Mandatory skill for tech roles

What is a Database?

- A database is an **organized** collection of data
- Data is stored in **tables**
- Each table has:

- Rows (records)
- Columns (attributes)



The diagram shows a table with four columns and three rows. A horizontal arrow labeled 'Columns' points from the first column to the last. A vertical arrow labeled 'Rows' points from the first row to the last. The table itself is a 4x3 grid with the following data:

ID	Name	Age	Course
1	John	20	Math
2	Asha	19	History
3	David	21	Physics

- **Real-Life Example:**
 - Student Database
 - Table: *Students*
 - Columns: ID, Name, Age, Course
 - Rows: Each student record

Why Databases?

- Fast data access
- Secure storage
- Data consistency
- Scalable

What is SQL?

- SQL is a standard language for relational databases
- SQL works with tables
 - Querying data
 - Inserting data
 - Updating data
 - Deleting data

SELECT

INSERT

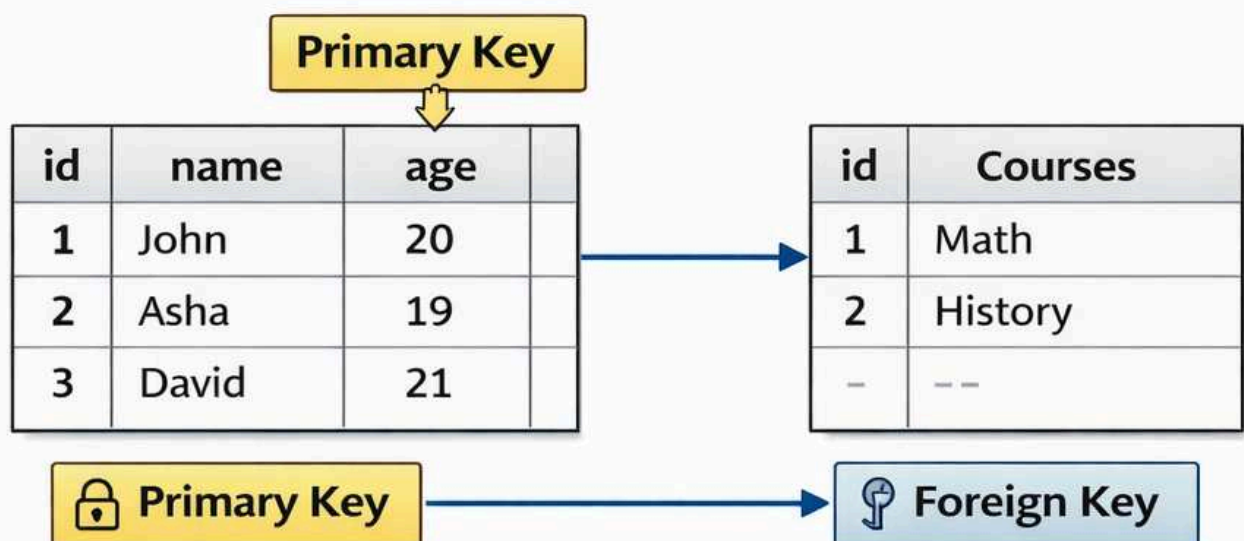
UPDATE

DELETE

Important Fact: **SQL** is not case-sensitive, but keywords are written in uppercase by convention.

Relational Databases

- Data stored in **related tables**
- Tables are connected using **keys**
- Ensures data integrity



- Each table has a **Primary Key**
- Tables are linked using **Foreign Keys**

SQL Data Types

SQL includes a variety of data types:

Numeric

- INT
- DECIMAL
- FLOAT

String

- VARCHAR
- CHAR
- TEXT

Date & Time

- DATE
- TIME
- TIMESTAMP

Boolean

- BOOLEAN

Types of SQL Commands

1. DDL (Data Definition Language)

- CREATE
- ALTER
- DROP
- TRUNCATE

2. DML (Data Manipulation Language)

- INSERT
- UPDATE
- DELETE

3. DQL (Data Query Language)

- SELECT

4. DCL (Data Control Language)

- GRANT
- REVOKE

5. TCL (Transaction Control Language)

- COMMIT
- ROLLBACK
- SAVEPOINT

CREATE TABLE

Purpose:

- Used to create a new table in the database

Syntax:



```
... CREATE TABLE students |:  
:  
: id INT,  
: name VARCHAR(50),  
:  
: age INT  
:  
: );
```

INSERT INTO

Purpose:

- Used to insert new data into an existing table

Syntax:



```
INSERT INTO table_name (column1, column2,..)  
VALUES (value1, value2, ..);
```

Example:

```
INSERT INTO students (id, name, age)  
VALUES (1, "John", 20);
```

SELECT

Purpose:

- Used to retrieve data from the database

Syntax:



```
SELECT column1, column2, ...  
FROM table_name  
  
[WHERE condition];
```

Example:

```
SELECT id, name, age  
FROM students;
```

WHERE

Purpose:

- Used to filter records based on specified conditions

Syntax:



```
SELECT column1, column2, ...  
FROM table_name  
[WHERE condition];
```

Example:

```
SELECT *  
FROM students;  
[WHERE age > 18;
```

OPERATORS



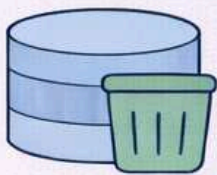
Arithmetic:

+ + /
+ -
* *
/



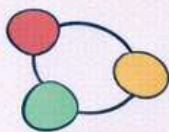
Comparison:

=
!=
>
<
>=



Using SELECT *

SELECT *
FROM sales;



Wrong Joins

AND
OR
NOT



Ignoring Indexes

ORDER BY

Purpose:

- Used to sort the result set in ascending or descending order

Syntax:



```
SELECT column1, column2, ...  
FROM table_name  
  
ORDER BY column1 [ASC | DESC];
```

Example:

```
SELECT id, name, age  
FROM students  
ORDER BY age DESC;
```

LIMIT & OFFSET

Purpose:

- Used to limit the number of rows returned by a query and to specify an offset for the starting point

Syntax:



```
SELECT column1, column2, ...  
FROM table_name  
  
[LIMIT number_of_rows]  
[OFFSET number_of_rows];
```

Example:

```
SELECT *  
FROM students  
  
LIMIT 5 OFFSET 10;
```

AGGREGATE FUNCTIONS

Purpose:

- Used to perform calculations on multiple rows of data and return a single value

Syntax:



```
SELECT AGG_FUNCTION(column_name)
FROM table_name;
```

- COUNT(), SUM(), • AVG(), • MAX()

Examples:

- COUNT()
- SUM()
- AVG()
- MIN()
- MAX()

```
SELECT COUNT(*) FROM orders;
```

```
SELECT SUM(price) FROM sales;
```

```
SELECT AVG(score) FROM tests;
```

```
SELECT MIN(age) FROM people;
```

```
SELECT MAX(age) FROM people;
```

GROUP BY

Purpose:

- Used to group rows that have the same values in specified columns and perform aggregate functions on them

Syntax:



```
SELECT column1, AGG_FUNCTION(column2)
FROM table_name;
GROUP BY column1;
```

- COUNT(), SUM(), • AVG(), • MIN(), • MAX()

Example:

- SELECT
- SUM()
- AVG()
- MIN()
- MAX()

```
SELECT category, COUNT(*)
FROM products
GROUP BY category;
```

HAVING

Purpose:

- Used to filter grouped rows based on a condition with aggregate functions

Syntax:



```
SELECT column1, AGG_FUNCTION(column2)
FROM table_name
GROUP BY column1
HAVING condition;
```

Example:

- SELECT
- SUM()
- AVG()
- MIN()

```
SELECT category, COUNT(*)
FROM products
GROUP BY category
HAVING COUNT(*) > 10;
```

JOINS

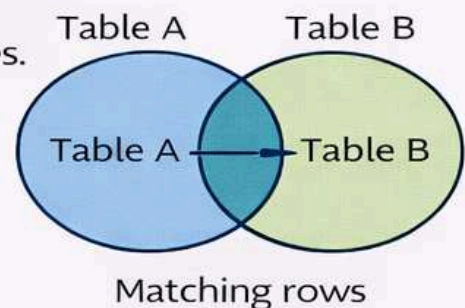
Purpose:

- Used to combine rows from two or more tables based on a related column

INNER JOIN

Returns only matching rows between the tables.

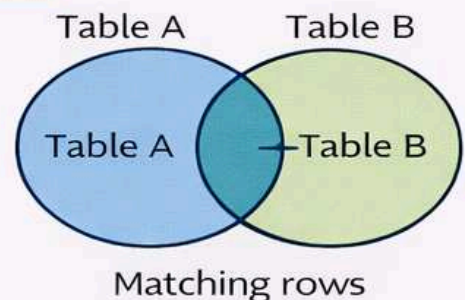
```
SELECT * FROM tableA
INNER JOIN tableB
ON tableA.id = tableB.id;
```



LEFT JOIN

Returns all rows from the left table and matching rows from the right.

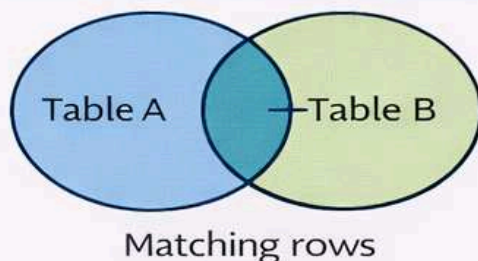
```
SELECT * FROM tableA
LEFT JOIN tableB
ON tableA.id = tableB.id;
```



RIGHT JOIN

Returns all rows from the right table and matching rows from the left.

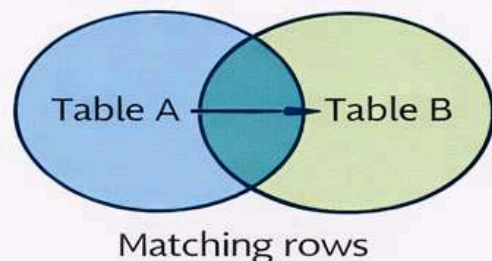
```
SELECT * FROM tableA
RIGHT JOIN tableB
ON tableA.id = tableB.id;
```



FULL JOIN

Returns all rows when there is a match in either left or right table.

```
SELECT * FROM tableA
FULL JOIN tableB
ON tableA.id = tableB.id;
```



INNER JOIN vs LEFT JOIN

orders

order_id	customer_id	product
1	101	Laptop
2	103	Tablet
3	102	Phone
4	105	Monitor

customers

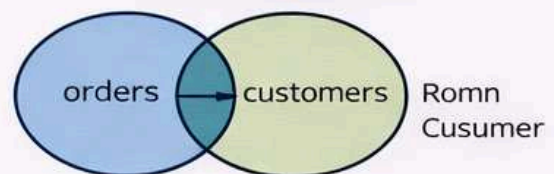
customer_id	name
101	Alice
102	Bob
103	Charlie
104	David

INNER JOIN



```
SELECT orders.order_id,  
customers.name, orders.product  
FROM orders  
INNER JOIN customers  
ON orders.customer_id = customers.  
customer_id;
```

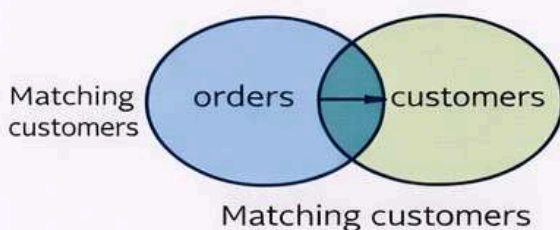
LEFT JOIN



```
SELECT orders.order_id,  
customers.name, orders.produ  
FROM orders  
LEFT JOIN customers  
ON orders.customer_id = customers-  
customer_id;
```

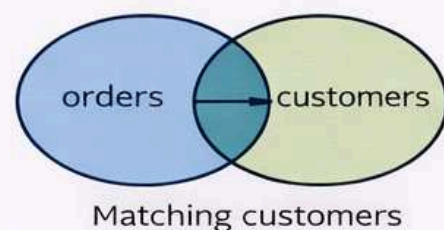
INNER JOIN

order_id	name	product
1	Alice	Laptop
2	Charlie	Tablet
3	Bob	Phone
4		



LEFT JOIN

order_id	name	product
1	Alice	Laptop
2	Charlie	Tablet
3	NULL	Phone
4	NULL	Monitor



SUBQUERIES

Purpose:

- Used to nest a query inside another query to retrieve data based on the result of the inner query

Syntax:



```
SELECT column1, column2, ...  
FROM table  
WHERE column_name OPERATOR  
(SELECT column FROM table WHERE condition);
```

- =, !=, >, <, >=, <=, IN, NOT IN...

Example:

INNER JOIN

order_id	name	product
1	Alice	Laptep
2	Charlie	Tablet
3	Bob	Phone



```
SELECT AVG(score)  
FROM students
```

LEFT JOIN

order_id	name	product
1	Alice	Laptep
2	Charlie	Tablet
3	DoD	Phone
4	NULL	Monitor



```
SELECT AVG(score)  
FROM students
```

CONSTRAINTS

PRIMARY KEY Ensures unique identification of each row

```
CREATE TABLE students (  
  student_id INT PRIMARY KEY,  
  name VARCHAR(50).  
);
```

FOREIGN KEY Ensures referential integrity between tables

```
CREATE TABLE orders (  
  order_id INT,  
  FOREIGN KEY (customer_id) REFERENCES customers(customer_id)  
);
```

UNIQUE Ensures all values in a column are unique

```
CREATE TABLE products (  
  product_id INT PRIMARY KEY,  
  sku VARCHAR(20) UNIQUE  
);
```

NOT NULL Ensures a column cannot have NULL values

```
CREATE TABLE users (  
  user_id INT PRIMARY KEY,  
  email VARCHAR(100) NOT NULL  
);
```

INDEXES

Purpose: Used to improve query performance by creating a quick lookup to rows in a table based on the values of specified columns

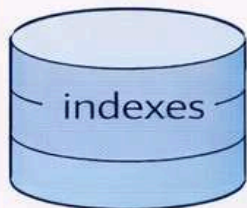
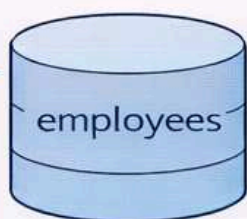
Syntax:



```
CREATE INDEX idx_last_name  
ON table_name  
(column_name);
```

Example:

```
CREATE INDEX idx_last_name ON employees  
(last_name);
```



idx_last_name	
Anderson	Anderson
Garcia	Garcia
Smith	
Wilson	Wilson

```
CREATE INDEX idx_last_name  
ON employees (last_name);
```

NORMALIZATION

UNNORMALIZED FORM

student_id	name	courses	advisor
1	Alice	Math, History	Bob
2	Bob	Math, English	John
3	Charlie	Science, History	Bob



1NF (First Normal Form)

student_id	name	course	advisor
1	Alice	Math	Bob
2	Bob	History	John
3	Charlie	English	Bob

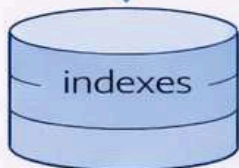


2NF (Second Normal Form)

student_id	name	student_id	advisor
1	Alice	1	Bob
2	Bob	2	John
3	Charlie	3	Bob



3NF (Third Normal Form)



student_id	name
1	Alice
2	Bob
3	Charlie

student_id	advisor
1	Bob
2	John
3	Science

student_id	course
1	Math
1	History
2	Math
2	English
3	Science

student_id	advisor_id	advisor
1	1	Bob
2	2	John

course_id	course	course
1	1	Math
2	2	History
3	4	English
4	4	Science

TRANSACTIONS

Properties:

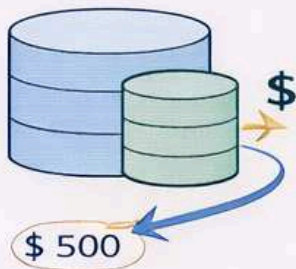
- Transactions in a database are characterized by four properties, collectively known as **ACID**:

A Atomicity Ensures that all parts of a transaction are completed; if any part fails, the entire transaction is aborted.

C Consistency Ensures that a transaction brings the database from one valid state to another, maintaining data integrity.

I Isolation Ensures that transactions are securely and independently performed without interference.

D Durability Ensures that once a transaction is committed, it remains permanently in the system, even in the event of a failure.



BEGIN:

```
UPDATE accounts SET balance = balance - 500  
WHERE account_id = 1;
```

```
UPDATE accounts SET balance = balance + 500  
WHERE account_id = 2;
```

```
COMMIT;
```



student_id	name
1	Alice
2	Bob
3	Charlie

student_id	advisor_id	advisor
1	1	Bob
2	2	John
3	4	Science

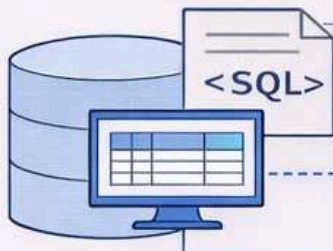
A Atomicity **C** Consistency **I** Isolation **D** Durability

ACID = Atomicity, Consistency, Isolation, Durability

VIEWS

Purpose:

- A view is a virtual table that displays data from one or more tables; used to simplify complex queries and enhance security.

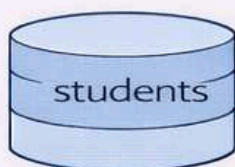


```
CREATE VIEW View_name AS  
SELECT column1, column2, ...  
FROM table  
WHERE condition;
```

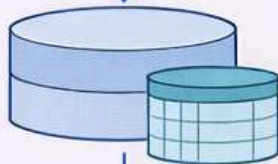


Example:

```
CREATE VIEW high_scores AS  
SELECT name, score  
FROM students  
WHERE score > 85;
```



student_.	id	name	age	score	score
1	1	Sarah	20	92	92
2	2	David	21	91	90
3	3	Bob	20	81	81
4	5	Emily	19	78	



```
CREATE VIEW high_scores AS  
SELECT name, score  
FROM students  
WHERE score > 85;
```

high_scores

A view based on the "students" table where score > 85.

A Atomicity

C Consistency

I Isolation

D Durability

STORED PROCEDURES

Purpose:

- Reusable SQL logic
- Stored inside the database



```
CREATE PROCEDURE procedure_name  
    (param1 datatype, param2 datatype, ... )  
BEGIN  
    SQL statements;
```

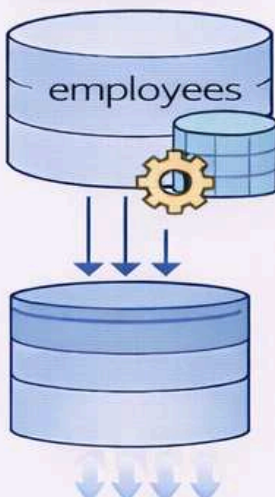


Example:

```
CREATE PROCEDURE add_bonus  
    IN emp_id INT,  
    IN bonus DECIMAL(10, 2)  
BEGIN  
    UPDATE employees  
        SET salary = salary + bonus  
        WHERE employee_id = emp_id;  
END;
```



Example:



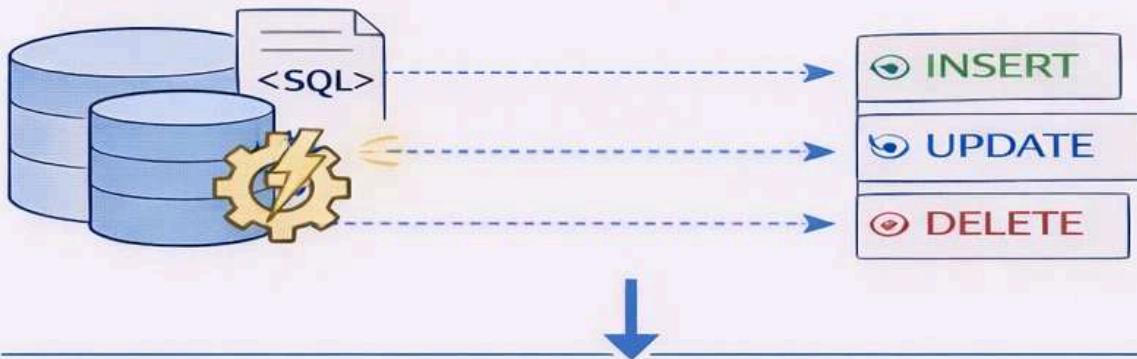
```
CREATE PROCEDURE add_bonus  
    IN emp_id INT,  
    IN bonus DECIMAL(10, 2)  
BEGIN  
    UPDATE employees  
        SET salary = salary + bonus  
        WHERE employee_id = emp_id;  
END;
```

A view based on the "students" table where score > 85.

TRIGGERS

Purpose:

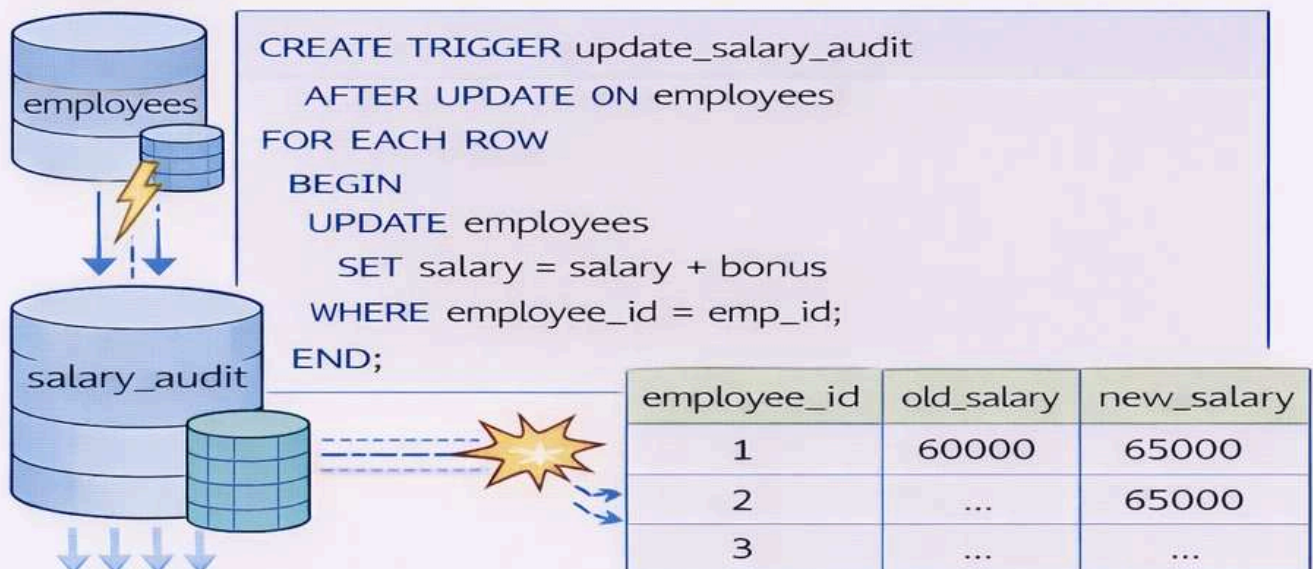
- Automatic execution of SQL code
- Runs when an INSERT, UPDATE, or DELETE operation occurs on a table



Syntax:

```
<SQL> CREATE TRIGGER trigger_name
        AFTER|BEFORE INSERT|UPDATE|DELETE
        ON table
        FOR EACH ROW
        BEGIN
        END;
```

Example:



A view based on the "students" table where score > 85.

SQL PERFORMANCE TIPS

Purpose:



Use Indexes Wisely

- Create indexes on columns that are frequently filtered or joined in queries

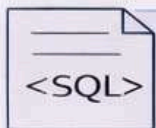


Avoid SELECT *.

- Specify the needed columns instead of selecting all columns with SELECT *.



Example:



```
CREATE TRIGGER update_salary_audit
  AFTER UPDATE ON employees
  FOR EACH ROW
  BEGIN
    UPDATE employees
      SET salary = salary + bonus
      WHERE employee_id = emp_id;
  END;
```

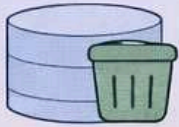


```
CREATE TRIGGER update_salary_audit
  AFTER UPDATE ON employees
  FOR EACH ROW
  UPDATE employees
    SET salary = salary + bonus
    WHERE employee_id = emp_id;
END;
```

employee_id	old_salary	new_salary
1	60000	65000
2	...	...

SQL FOR INTERVIEWS

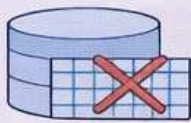
DELETE vs TRUNCATE vs DROP



- DELETE** • Deletes specific rows from a table.
- Can be rolled back (if used inside a transaction).



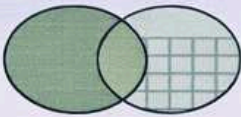
- TRUNCATE** • Removes all rows from a table quickly.
- Usually cannot be rolled back.



- DROP** • Deletes the entire table from the database along with its structure.
- Cannot be rolled back.

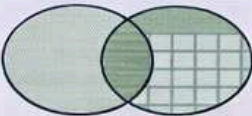
JOIN Types

- INNER JOIN** • Returns rows with matching values in both tables.



- Returns all rows from the left table, and matching rows from the right table.

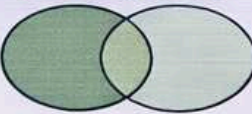
RIGHT JOIN



- Returns all rows from the right table, and matching rows from the left table.

- Returns all rows when there is a match in either table.

FULL JOIN



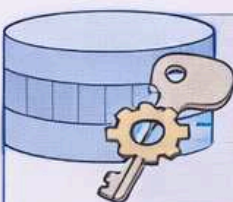
- Returns all rows when there is a match in either table.

JOIN Types

- ≡ **WHERE:** Filters rows before grouping.

- Ⓜ **HAVING:** Filters groups after the **GROUP BY** clause.
Works on grouped rows.

Primary Key vs Unique Key



- **Primary Key:** Uniquely identifies each row and cannot contain NULL values.
- **Unique Key:** Enforces uniqueness across a column but can have NULL values.

COMMON SQL MISTAKES



Missing WHERE Clause

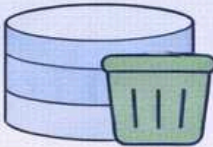


```
UPDATE employees SET salary = 0; -OOPS!
```

- Results in deleting or updating all rows.



Using SELECT *

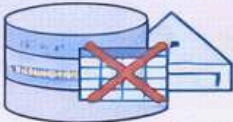


- Fetches all columns, even those not needed.

```
SELECT * FROM sales;
```



Wrong Joins

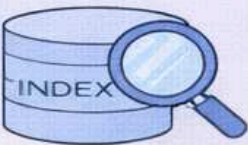


- Results in missing or incorrect data.

```
SELECT emp.name, dep.name  
FROM employees emp  
JOIN departments dep  
ON emp.id = dep.id; — PROBLEM: Missing rows!
```



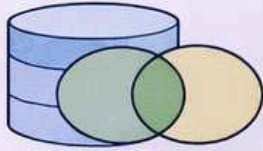
Ignoring Indexes



- Causes slow query performance.

```
SELECT * FROM orders  
WHERE customer_id = 1; — SLOW: No index!
```

SQL ROADMAP AHEAD



Advanced Joins: _____

- Learn about CROSS JOIN, NATURAL JOIN, and SELF JOIN.



Window Functions: _____



- Master ROW_NUMBER(), RANK(), DENSE_RANK(), and more.



Analytics SQL _____

- Use SQL for data analysis and business intelligence.



Analytics SQL _____

- Use SQL for data analysis and business intelligence.



Optimization Techniques _____

- Improve query performance and learn best practices.



SQL mastered = Data unlocked.

Practice daily. Build confidence.

Query like a pro.
